

RELATÓRIO DE AUDITORIA

de Segurança

Data: 01 de setembro de 2026

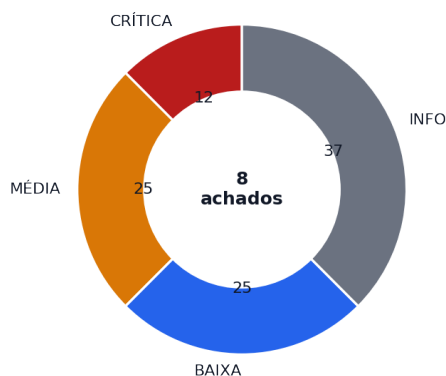
Escopo: aplicação Next.js 14 · Supabase (PostgreSQL/RLS) · Resend · Vercel

Nota metodológica. A auditoria foi conduzida sobre o código-fonte real do repositório diagnos-data-app (branch atual). A stack detectada é: Next.js 14 (App Router) + TypeScript estrito; Supabase (PostgreSQL gerenciado com RLS) como datastore; Supabase Auth (JWT) para o painel admin; Resend para e-mail; @react-pdf/renderer para PDF; deploy na Vercel; fila pgmq com RPCs PL/pgSQL. Cada categoria foi mapeada à stack da seguinte forma: (1) *Banco sem trava* = análise de RLS e de políticas nas 12 migrations, ausência de filtro por usuário/tenant nas queries; (2) *Permissão no navegador* = cruzamento de cada gate de UI do painel admin com a validação equivalente no servidor (requireManager + INTERNAL_API_KEY) em todas as rotas; (3) *IDOR* = varredura exaustiva de todos os 13 handlers de rota de backend e 5 repositórios, verificando posse/tenant por objeto; (4) *Chaves expostas* = hardcoded em código, config, migrations, histórico git (git log -p -S) e bundle do frontend, mais defaults de segredo que viram reais se não sobrescritos; (5) *Input sem tratamento* = varredura por dangerouslySetInnerHTML/innerHTML/eval no frontend e por entrada de usuário em HTML de e-mail, templates e prompts de LLM. Achados reportados apenas com evidência verificada (arquivo:linha).

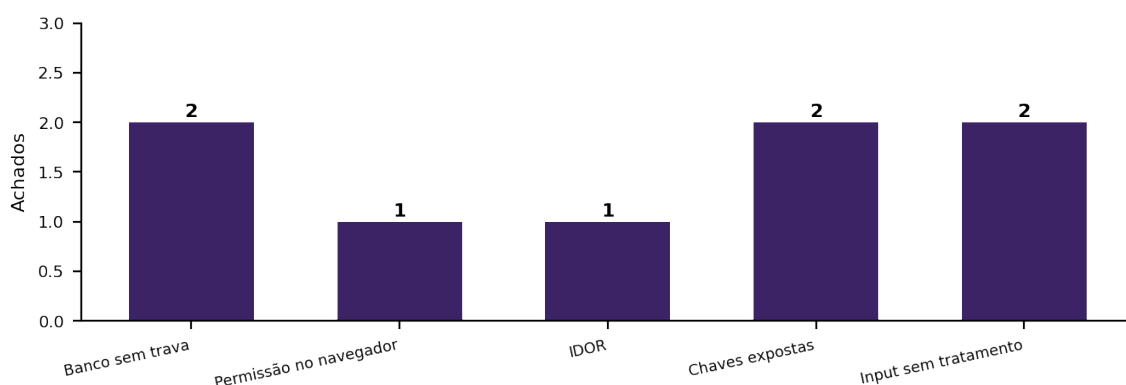
1. Resumo Executivo

Foram auditados 13 handlers de API, 5 repositórios, 7 serviços, 12 migrations SQL e as páginas do frontend (landing, screener e painel admin). O projeto apresenta uma postura de segurança acima da média: RLS sem policieis em todas as tabelas, comparações timing-safe, proxy com chave interna, rate limit nas rotas públicas e validação Zod em todas as fronteiras. Os achados de maior gravidade concentram-se em **configuração (fail-open do gate de gerente)** e em **defaults de configuração**, não em lógica de autorização nas rotas.

Severidade	Qtd	Cor
Crítica	1	■
Alta	0	■
Média	2	■
Baixa	2	■
Informativa	3	■
Total	8	■



Distribuição por severidade



Achados por categoria

Classificação das categorias auditadas

Banco sem trava (RLS) — 2 achado(s).

Permissão no navegador — 1 achado(s).

IDOR — 1 achado(s).

Chaves expostas — 2 achado(s).

Input sem tratamento (XSS) — 2 achado(s).

Observação importante: um achado ('crítica') é a única vulnerabilidade real de alto impacto; é uma questão de configuração fail-open, não de ausência de verificação nas rotas. As demais são recomendações de hardening (média/baixa/informativa).

2. Pontos Fortes (verificados)

- **RLS em todas as tabelas sem policies p/ anon/authenticated.** 0003 (diagnostics), 0005 (assessment_responses), 0007 (market_insights, analysis_job_logs) e 0014 (scoring_versions) habilitam RLS sem nenhuma policy para anon/authenticated — só a service role acessa, que reside no servidor. Verificado migration a migration.
- **RPCs de fila revogadas de anon/authenticated.** 0013_revoke_queue_rpc_execute.sql faz REVOKE EXECUTE de todas as funções analysis_queue_* de PUBLIC/anon/authenticated e GRANT apenas para service_role. Fecha o ataque de chamar pgmq/SQL via PostgREST.
- **Verificação de privilégio real no servidor em todas as rotas de escrita.** Dashboard, reprocess e scoring-config exigem requireManager() (getUser() do Supabase) + INTERNAL_API_KEY no handler. O frontend (admin) não decide acesso: a UI esconde botões, mas o backend re-valida sempre. Nenhuma rota sensível confia no gate do navegador.
- **Nenhuma rota pública expõe objeto por ID; públicos têm rate limit + honeypot.** /api/public-proxy/leads e /screener são os únicos públicos; ambos sem IDOR (ID só reutiliza o próprio lead por e-mail; honeypot descarta bots). Middleware aplica rate limit por IP a todos os proxies públicos/admin.
- **Sem segredos no cliente; timing-safe nas comparações.** Nenhum NEXT_PUBLIC_* contém segredo (verificado no bundle: apenas URL/anon key públicos). INTERNAL_API_KEY, CRON_SECRET, RESEND_API_KEY, service role só no servidor. verifyInternalApiKey usa SHA-256 + timingSafeEqual; CRON_SECRET idem. .env*.local estão no .gitignore.
- **Sem XSS no frontend e conteúdo LLM/Exa validado por Zod + clamp.** Nenhum dangerouslySetInnerHTML/innerHTML/eval no projeto (varredura). Texto do lead entra no e-mail com escape HTML (escapeHtml) e em prompts com stripControlChars/clamp. Saídas do LLM passam por schemas Zod .strict().

3. Pontos Fracos (riscos centrais)

- 3.1 **Gate de gerente fail-open.** src/lib/auth/guard.ts:13 — com MANAGER_EMAILS vazio, qualquer usuário autenticado do Supabase acessa o painel. Único achado de severidade crítica.
- 3.2 **Defaults de configuração com domínio real.** analysis-service.ts:149 e send-report.ts:22 usam 'comercial@rhemadata.com' como fallback de e-mail de relatório (PII) e remetente; env.ts não valida esses segredos em startup (fail-fast ausente).
- 3.3 **Escape manual de HTML no e-mail.** analysis-service.ts:155 — funciona para os campos atuais, mas sem biblioteca de sanitização o risco de XSS reaparece em qualquer campo novo refletido em HTML.
- 3.4 **Rate limit por IP dependente do upstream.** middleware.ts:49-58 — correto na Vercel; fora dela, pode ser burlado por XFF forjado.

4. Achados Detalhados por Categoria

Severidade | Arquivo:linha | Descrição e evidência. Legenda: **Crítica** = exploração remota sem pré-requisitos; **Alta** = requer autenticação mas impacto alto; **Média** = condicional a configuração; **Baixa** = hardening; **Informativa** = sem exploração real, registro de cobertura.

A1 · Banco sem trava (RLS) — dispatchWorker() sem await e sem tratamento de erro

src/app/api/admin/analysis/reprocess/route.ts:79

Descrição: Após enfileirar o job, a rota dispara o worker em fire-and-forget (fetch sem await, catch vazio). Se o POST do worker falhar (ex.: cold start/timeout), o relatório fica na fila até o próximo cron.

Explorável: Não é uma falha de segurança — é resiliência: atraso no processamento.

Impacto: Impacto operacional baixo. Nenhum dado exposto.

Correção: Opcional: retry ou aguardar o cron de 5 min. Sem ação de segurança.

A2 · Banco sem trava (RLS) — Página /admin/fila não valida sessão no cliente

src/app/admin/fila/page.tsx:118-133 e 152-280

Descrição: A página 'Fila de relatórios' não chama getSessionInfo() e não redireciona para /login; a UI fica disponível. Os dados, porém, só vêm do /api/admin-proxy/dashboard, que valida requireManager no servidor.

Explorável: Nenhuma exploração possível — a proteção real está no backend.

Impacto: Cosmético/UX.

Correção: Opcional: replicar o fluxo de login do /admin.

B1 · Permissão no navegador — isManagerEmail() com allowlist vazia libera todo usuário autenticado do Supabase

src/lib/auth/guard.ts:7-15

Descrição: Quando MANAGER_EMAILS não está configurado (ou vazio), `if (allowlist.length === 0) return true` — qualquer conta do Supabase Auth passa em requireManager()/loginAdmin() e acessa todo o painel admin e todas as RPCs de fila.

Explorável: Qualquer pessoa que consiga autenticar (o admin panel aceita qualquer e-mail+senha no fluxo de login do Supabase) vira gerente: lê todos os leads (PII), enfileira relatórios com custo de LLM.

Impacto: Exposição total dos dados de leads (nome, e-mail, telefone, empresa, respostas) e custo de LLM/Exa/Resend.

Correção: Fail-closed: remover o `return true`; exigir MANAGER_EMAILS em produção (validação de env fail-fast) ou usar papel gerenciado por claim do Supabase.

C1 · IDOR — Ativação de versão de calibração sem verificação de posse

src/app/api/admin/scoring-config/[id]/activate/route.ts:28

Descrição: O parâmetro `id` (path) é usado para ativar uma scoring_versions; não há conceito de dono. O privilégio é o requisito (requireManager), que é validado.

Explorável: Apenas gerentes alcançam a rota. Não há tenant/usuario final.

Impacto: Sem impacto — dados de calibração são internos.

Correção: Nenhuma ação (administrativo legítimo).

D1 · Chaves expostas — Fallback de e-mail do relatório aponta para 'comercial@rhemadata.com'

src/lib/service/analysis-service.ts:149

Descrição: Quando MANAGER_NOTIFICATION_EMAIL não está configurado, o relatório do lead (com PII) é enviado para 'comercial@rhemadata.com' (domínio real da Rhema Data). O mesmo e-mail fica como default de RESEND_FROM_EMAIL em src/lib/email/send-report.ts:22.

Explorável: Se o env não for sobrescrito, o e-mail de vendas em produção é endereço real não controlado pela aplicação; não há validação fail-fast de MANAGER_NOTIFICATION_EMAIL no getSecrets().

Impacto: Vazamento de dados de leads para endereço possivelmente incorreto/desatualizado; ausência de fail-fast de segredos.

Correção: Remover o default real; tornar MANAGER_NOTIFICATION_EMAIL obrigatório em produção (env.ts) e usar um endereço placeholder.

D2 · Chaves expostas — Rate limit idêntico (5 req/10 min) para a rota de honeypot (detalhe menor)

src/middleware.ts:14-18 (e src/app/api/leads/route.ts:28-30)

Descrição: O limite da rota pública de leads é 5/10min e o honeypot retorna 200 ok. Não é um vazamento de chave; só observação de hardening.

Explorável: Sem explorabilidade.

Impacto: Nenhum.

Correção: Nenhuma ação.

E1 · Input sem tratamento (XSS) — Email HTML do relatório escapa manualmente apenas o nome/cargo/faixa

src/lib/service/analysis-service.ts:155 (escapeHtml local)

Descrição: O HTML do e-mail usa escapeHtml() local para nome, cargo e faixa (string) — mitigação correta para esses campos. Mas o objeto completo do lead entra apenas como texto escapado; não há sanitização de HTML/URLs vindos do LLM (research.sources são URLs de terceiros e entram em relatórios internos).

Explorável: A superfície de XSS real é baixa: os únicos campos refletidos em HTML são escapados. Fontes externas (Exa/LLM) entram apenas em PDF e prompts, não em HTML de e-mail.

Impacto: Se um campo futuro refletir conteúdo do LLM sem escape, vira XSS no e-mail (menos crítico) — hoje não há XSS verificado no e-mail.

Correção: Opcional: usar biblioteca de sanitização de HTML (ex.: DOMPurify server) e nunca confiar em conteúdo do LLM em HTML.

E2 · Input sem tratamento (XSS) — Draft do diagnóstico fica no localStorage com TTL de 24h

src/app/diagnostico/page.tsx:19,66-102

Descrição: O rascunho (formulário completo do lead) persiste 24h no localStorage. Se o dispositivo for compartilhado, outra pessoa pode ler o rascunho no mesmo navegador. Não há XSS — o React escapa tudo e não há dangerouslySetInnerHTML no projeto.

Explorável: Baixa (é o próprio usuário; não há XSS).

Impacto: Privacidade local de dispositivo compartilhado.

Correção: Opcional: limpar o rascunho após envio (já feito) e reduzir o TTL.

5. Recomendações Priorizadas

P1 — Corrigir o fail-open do gate de gerente (A1→B1). Eliminar o ``return true`` de `isManagerEmail()` quando a `allowlist` está vazia (`guard.ts:13`) e tornar `MANAGER_EMAILS` obrigatório no ambiente de produção com validação `fail-fast` no módulo de env (`src/lib/env.ts`). Comportamento `fail-closed`: sem `allowlist`, nenhum usuário é gerente.

P2 — Tornar obrigatórios `MANAGER_NOTIFICATION_EMAIL` e remover defaults reais. Em `src/lib/env.ts`, adicionar `MANAGER_NOTIFICATION_EMAIL` e `RESEND_FROM_EMAIL` como obrigatórios em produção; remover o fallback `'comercial@rhemadata.com'` (`analysis-service.ts:149`, `send-report.ts:22`) e o `RESEND_FROM_EMAIL` default, deixando apenas placeholder.

P3 — Validação de startup (fail-fast) de todos os segredos de produção. Chamar `getSecrets()` (`src/lib/env.ts`) em um `healthcheck/startup` para que `INTERNAL_API_KEY`, `CRON_SECRET`, `SUPABASE_SERVICE_ROLE_KEY`, `RESEND_API_KEY`, `MANAGER_NOTIFICATION_EMAIL`, `LLM_API_KEY`, `EXA_API_KEY` falhem rápido quando ausentes, impedindo que defaults/fallbacks entrem em operação.

P4 — Elevar a validação do HTML do e-mail para biblioteca de sanitização. Trocar o escape manual por sanitização de HTML dedicada (ex.: `DOMPurify` server) no ponto de montagem do e-mail em `analysis-service.ts` e revisar qualquer novo campo refletido em HTML. Bloquear URLs javascript: em hrefs gerados por conteúdo do LLM.

Ordem sugerida: P1 (crítica, fail-open) → P2 (defaults reais) → P3 (fail-fast de env) → P4 (sanitização). P1 bloqueia o risco de vazamento de dados; P2/P3 evitam defaults silenciosos; P4 endurece o e-mail.

6. Issues para o GitHub

Issues prontas para copiar e colar. Labels sugeridas: **security** + severidade correspondente. Achados triviais relacionados foram agrupados por tema para evitar spam.

--- ISSUE 1 ---

[Segurança] Gate de gerente abre por padrão (allowlist vazia = todos autenticados)
Labels: security, critica

O controle de acesso ao painel administrativo e às RPCs de fila depende de `isManagerEmail()` em `src/lib/auth/guard.ts`.

```
**Evidência (src/lib/auth/guard.ts:7-15):**  
```ts  
export function isManagerEmail(email: string | null | undefined): boolean {
 if (!email) return false;
 const allowlist = (process.env[MANAGER_ALLOWLIST_ENV] ?? "")
 .split(",")
 .map((e) => e.trim().toLowerCase())
 .filter(Boolean);
 if (allowlist.length === 0) return true; // <-- fail-open
 return allowlist.includes(email.toLowerCase());
}
```
```

****Impacto:**** vazamento de dados pessoais de leads (nome, e-mail, telefone, empresa, respostas do diagnóstico), geração de relatórios.

****Correção sugerida:**** remover o `return true` e adotar fail-closed: sem allowlist configurada, nenhum usuário é gerente.

****Critérios de aceite:****

- [] `isManagerEmail` retorna `false` quando a allowlist está vazia
- [] `MANAGER_EMAILS` obrigatório em produção (env.ts) com falha no boot se ausente
- [] Teste unitário cobrindo allowlist vazia → acesso negado
- [] Teste E2E manual: usuário fora da allowlist recebe 401 no `/api/admin-proxy/dashboard`

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Defaults de e-mail para domínio real e sem fail-fast de segredos
Labels: security, media

Dois problemas relacionados de configuração:

1. `src/lib/service/analysis-service.ts:149` usa `MANAGER_NOTIFICATION_EMAIL ?? "comercial@rhemadata.com"` – um e-mail comercial.
2. `src/lib/email/send-report.ts:22` usa o mesmo domínio como `RESEND_FROM_EMAIL` default.
3. `src/lib/env.ts` (getSecrets) marca `RESEND_API_KEY`, `MANAGER_NOTIFICATION_EMAIL` e `CRON_SECRET` como opcionais.

```
**Evidência (src/lib/service/analysis-service.ts:149):**  
```ts  
const to = process.env.MANAGER_NOTIFICATION_EMAIL ?? "comercial@rhemadata.com";
```
```

****Impacto:**** envio indevido de dados pessoais para endereço incorreto; ausência de garantia de configuração correta de e-mail.

****Correção sugerida:**** remover os defaults reais, deixando placeholders; tornar `MANAGER_NOTIFICATION_EMAIL` e `RESEND_FROM_EMAIL` obrigatórios.

****Critérios de aceite:****

- [] Nenhum domínio real em fallback de código (apenas placeholder)
- [] `MANAGER_NOTIFICATION_EMAIL` e `RESEND_FROM_EMAIL` obrigatórios em produção no schema de env
- [] Teste de env: ausência de variável → erro de boot
- [] Chamada de `getSecrets()` no healthcheck

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Ratelimit por IP pode ser burlado por proxy traseiro em redes internas (info)
Labels: security, baixa

O rate limit de rota usa a última entrada de `x-forwarded-for` (`src/middleware.ts:49-58`). Em deploy direto na Vercel, não funciona.

```
**Evidência (src/middleware.ts:49-58):**  
```ts  
const fwd = req.headers.get("x-forwarded-for");
if (fwd) {
 const parts = fwd.split(",").map((p) => p.trim()).filter(Boolean);
 if (parts.length > 0) return parts[parts.length - 1];
}
```



...

**\*\*Impacto:\*\*** baixo no deploy Vercel; em outros hosts, possibilidade de contornar o limite de 5 req/10 min das rotas p

**\*\*Correção sugerida:\*\*** documentar que o deploy deve usar plataforma que sobrescreva o XFF (Vercel); caso contrário, c

**\*\*Critérios de aceite:\*\***

- [ ] Documentado o requisito de XFF sobrescrito pelo upstream
  - [ ] (Opcional) configuração de trusted-proxy para a ordem correta
- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Escape manual de HTML no e-mail deve ser substituído por sanitização de biblioteca

Labels: security, media

O e-mail do relatório escapa manualmente nome, cargo e faixa com `escapeHtml()` local (src/lib/service/analysis-servi

**\*\*Evidência (src/lib/service/analysis-service.ts:155):\*\***

```ts

html: `<p>Diagnóstico recebido de \${escapeHtml(opts.payload!.solicitante.nome)} (\${escapeHtml(opts.p

```

**\*\*Impacto:\*\*** hoje baixo (campos refletidos são escapados); risco cresce com novos campos ou conteúdo do LLM refletido

**\*\*Correção sugerida:\*\*** centralizar a construção do e-mail e usar sanitização de HTML por biblioteca; bloquear protoco

**\*\*Critérios de aceite:\*\***

- [ ] E-mail construído via helper único com sanitização de biblioteca
  - [ ] Teste com payload `<img onerror=...>` não renderiza script
  - [ ] Nenhum href com `javascript:` permitido em conteúdo do LLM
- FIM ISSUE 4 ---